



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS**

**A FINAL PROJECT REPORT
ON
SMART GLOVE FOR REAL TIME NEPALI SIGN LANGUAGE
TO SPEECH TRANSLATION**

SUPERVISED BY:
ASST. PROF. KHEM RAJ KOIRALA

SUBMITTED BY:

GANESH ROKAYA	PAS078BEI013
SANJOG SAPKOTA	PAS078BEI034
SANTOSH KUMAR BARAI	PAS078BEI035
UPENDRA RAJ JOSHI	PAS078BEI046

SUBMITTED TO:
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PASHCHIMANCHAL CAMPUS

May, 2026

**SMART GLOVE FOR REAL TIME NEPALI SIGN LANGUAGE TO SPEECH
TRANSLATION**

SUBMITTED BY:

GANESH ROKAYA	PAS078BEI013
SANJOG SAPKOTA	PAS078BEI034
SANTOSH KUMAR BARAI	PAS078BEI035
UPENDRA RAJ JOSHI	PAS078BEI046

SUPERVISED BY:

ASST. PROF. KHEMRAJ KOIRALA

**A MAJOR PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT FOR THE BACHELOR'S DEGREE IN
ELECTRONICS, COMMUNICATION AND INFORMATION ENGINEERING**

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PASHCHIMANCHAL CAMPUS

May, 2026

COPYRIGHT

The authors have agreed that the Library, Department of Electronics and Computer Engineering, Institute of Engineering, Pashchimanchal Campus may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisors who supervised the project work recorded herein or in their absence, by the Head of the Department wherein the project report was done. It is understood that the recognition will be given to the authors of this project and to the Department of Electronics and Computer Engineering, Pashchimanchal Campus, Institute of Engineering in any use of the material of this report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, Institute of Engineering, Pashchimanchal Campus and authors' written permission is strictly prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to:

Head of Department

Department of Electronics and Computer Engineering

Institute of Engineering, Pashchimanchal Campus

ACKNOWLEDGEMENT

We are deeply grateful to everyone who supported and guided us throughout this project. While the work required considerable effort on our part, it would not have been possible without the help and encouragement of many individuals and organizations. First and foremost, we would like to express our sincere gratitude to our supervisor, **Asst. Prof. Khem Raj Koirala**, for his continuous guidance, constant support, and valuable inputs throughout the course of this project. We would also like to thank **Asst. Prof. Hom Nath Tiwari**, Head of the Department of Electronics and Computer Engineering, for his constructive feedback and suggestions that helped us improve our work at every stage of development. We extend our gratitude to the **Department of Electronics and Computer Engineering, IOE, Pashchimanchal Campus**, for giving us the opportunity to undertake this major project as part of our academic curriculum. Finally, we would like to thank our colleagues and all those who generously offered their time and assistance whenever we needed it.

GANESH ROKAYA

SANJOG SAPKOTA

SANTOSH KUMAR BARAI

UPENDRA RAJ JOSHI

ABSTRACT

This project presents a wearable smart glove system developed to enable real-time translation of Nepali Sign Language gestures into spoken audio output, addressing the communication barrier faced by the deaf and hearing-impaired community in Nepal. The hardware prototype integrates eight flex sensors distributed across the finger joints and an MPU6050 inertial measurement unit mounted on an ESP32 microcontroller, which together capture both static finger-bending configurations and dynamic hand motion trajectories. Sensor signals are transmitted via Bluetooth to a host computer, where a dual-model inference pipeline classifies gestures in real time: a Random Forest classifier handles nine static gestures, including four Nepali alphabet signs and four numerical signs, while a two-layer Long Short-Term Memory network processes seven dynamic word-level gestures representing a basic self-introduction sequence. The Random Forest model achieved a test accuracy of 99.56%, and the LSTM model stabilized at approximately 94% validation accuracy, with an overall real-time system accuracy of 93.1% across 320 trials. Predicted gesture labels are mapped to pre-generated Nepali audio files produced using the Google Text-to-Speech library and played through a speaker, with average translation latencies of 0.5 seconds for static gestures and approximately 2 seconds for dynamic ones. While the current implementation is limited to a small vocabulary collected from a restricted user group, the results demonstrate the technical feasibility of sensor-based Nepali Sign Language recognition and establish a working foundation for future expansion toward larger vocabularies, embedded deployment, and community-scale data collection involving deaf users.

Keywords: Nepali Sign Language, Smart Glove, Gesture Recognition, Text-to-Speech

TABLE OF CONTENTS

COPYRIGHT	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF FIGURES	vi
LIST OF TABLE	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem statement	1
1.3 Objectives	2
2 LITERATURE REVIEW	3
3 METHODOLOGY	5
3.1 Requirement Analysis	5
3.1.1 Hardware Requirements	5
3.1.2 Software Requirements	6
3.2 Block Diagram	8
3.3 Hardware Interfacing	9
3.3.1 Interfacing of Flex Sensors	9
3.3.2 Interfacing of IMU	10
3.4 Sensor Calibration	11
3.5 Data Collection and Dataset Preparation	12
3.5.1 Data Collection From Hardware	12
3.5.2 Dataset Creation for Static Gestures	13
3.5.3 Dataset Creation for Dynamic Gestures	15
3.6 Model Training and Development	16
3.6.1 Random Forest for Static Gesture	16
3.6.2 LSTM for Dynamic Gesture	17
3.7 Real-Time Gesture Prediction Pipeline	19
3.8 Text-to-Speech Conversion	20
4 RESULT AND ANALYSIS	21
4.1 Real-Time Prediction Results	21

4.2	Analysis of Random Forest Model	21
4.3	Analysis of LSTM model	22
5	EPILOGUE	24
5.1	Challenges and Limitations	24
5.2	Future Enhancements	25
5.3	Conclusion	25
	APPENDIX A	26
A.1	Prototype Glove	26
A.2	PCB Schematic	27
A.3	Example Gestures	28
	REFERENCES	29

List of Figures

3.1	ESP32	5
3.2	Flex Sensor	6
3.3	MPU6050	6
3.4	Block Diagram	8
3.5	Flex Sensor Connection	9
3.6	Joint	9
3.7	Sensor Placement	9
3.8	Voltage across flex sensor vs Series Resistor	10
3.9	Interfacing of MPU6050	11
3.10	Flowchart for data acquisition process.	12
3.11	Flowchart of the static gesture dataset creation process.	14
3.12	Flowchart of the dynamic gesture dataset creation process.	15
3.13	Random Forest	16
3.14	LSTM	17
3.15	LSTM Training	18
3.16	Flowchart of the real-time gesture prediction pipeline.	19
4.1	Confusion Matrix of Random Forest Model	22
4.2	Confusion Matrix of LSTM Model	22
4.3	Learning Curve	23
4.4	LSTM Feature Importance Over Time	23
A.1	Prototype Glove	26
A.2	PCB Schematic	27
A.3	Example Gestures	28

List of Tables

3.1	Sample of Raw CSV Data Transmitted from the ESP32 Hardware During the Data Collection Process	13
3.2	Sample Records from the Static Gesture Dataset	14
4.1	Real-Time Prediction Results for Each Gesture (20 Trials Each)	21

LIST OF ABBREVIATIONS

ADC	Analog-to-Digital Converter
ASL	American Sign Language
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CSLR	Continuous Sign Language Recognition
CSV	Comma-Separated Values
DAC	Digital-to-Analog Converter
DL	Deep Learning
DMP	Digital Motion Processor
EDA	Electronic Design Automation
GPIO	General Purpose Input/Output
GPL	GNU General Public License
gTTS	Google Text-to-Speech
I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MCP	Metacarpophalangeal
ML	Machine Learning
NSL	Nepali Sign Language
OOB	Out-of-Bag
PCB	Printed Circuit Board
PIP	Proximal Interphalangeal
RAM	Random Access Memory
RF	Random Forest
RNN	Recurrent Neural Network
ROM	Read-Only Memory
SOC	System on Chip
SPI	Serial Peripheral Interface
STTN	Spatial-Temporal Transformer Network
TTS	Text-to-Speech
UART	Universal Asynchronous Receiver-Transmitter
WER	Word Error Rate

CHAPTER 1

INTRODUCTION

1.1. Background

Communication plays an important role in everyday life, allowing people to express their thoughts, needs, and emotions. For individuals with hearing and speech impairments, sign language serves as one of the primary ways of communication. In Nepal, Nepali Sign Language is widely used within the deaf community. However, a large portion of the population is unfamiliar with NSL, which creates difficulties in daily interactions such as in schools, hospitals, offices, public transportation, and social environments. Because of this communication gap, many deaf individuals often face challenges in expressing themselves clearly and independently.

With the rapid growth of embedded systems, wearable technology, and machine learning, several assistive systems have been developed to bridge the gap between sign language users and non-signers. Many existing sign language recognition systems rely on cameras and computer vision techniques to detect hand gestures. Although these systems can achieve good accuracy, they are often affected by lighting conditions, background noise, camera position, and limited mobility. Wearable sensor-based systems, on the other hand, provide a more portable and reliable alternative because they directly capture hand and finger movements through sensors attached to the glove.

1.2. Problem statement

For the deaf, hearing and speech impairment community in Nepal, NSL is a vital means of communication. However, most people in society do not understand NSL, making it difficult for sign language users to express themselves in everyday situations whether in schools, hospitals, workplaces, or public spaces. This communication gap often leads to frustration, exclusion, and a lack of independence. While some technologies exist for sign language translation, they are mostly built for other languages like ASL and are not suited for NSL's unique structure. Research and systems based on sensor technology are rare in this context, especially for capturing the complex, time-based nature of gestures. There is a clear need for a real-time system that can understand NSL gestures and translate them into speech, making communication more inclusive and accessible.

1.3. Objectives

The primary objectives of the proposed system are:

1. To build a wearable glove attached with flex sensors and IMUs.
2. To train machine learning model to recognize sign gesture and convert them into Nepali speech.

CHAPTER 2

LITERATURE REVIEW

Researchers have explored various machine learning and deep learning approaches, including CNNs, LSTM networks, transformer architectures, and sensor fusion techniques for recognizing static as well as dynamic hand gestures. This section reviews previous research works related to sign language recognition, wearable smart gloves, gesture classification techniques, and deep learning-based recognition systems that are relevant to this project.

Baral and Bhusal[1] proposed a vision-based Continuous Sign Language Recognition (CSLR) system for Nepali Sign Language (NSL) utilizing MediaPipe Holistic and a Spatial-Temporal Transformer Network (STTN). The proposed model achieved a Word Error Rate (WER) of 20.23% on a dataset containing 40 sentences while addressing challenges such as inconsistent gestures and variations in signing speed.

Aryal et al.[2] developed a wearable system for translating Nepali Sign Language gestures into speech using potentiometers, accelerometers, and gyroscope sensors. A Random Forest classifier trained on custom sensor data consisting of 9 words and 5 letters achieved 98% testing accuracy for real-time gesture-to-speech conversion.

Xia et al.[3] introduced a hybrid CNN-LSTM architecture for human activity recognition using mobile sensor data. The model effectively extracted both spatial and temporal features and achieved more than 95% accuracy on benchmark datasets while maintaining lower computational complexity compared to existing approaches.

Wu et al.[4] presented an Attention-based CNN-BiLSTM framework for dynamic hand gesture recognition using data gloves. By combining spatial-temporal feature extraction with attention mechanisms, the proposed system achieved an accuracy of 95.05% in recognizing complex gesture sequences such as seven-step handwashing activities.

Lee et al.[5] implemented a smart wearable system for American Sign Language (ASL) interpretation using six Inertial Measurement Units (IMUs) positioned on the fingers and hand. The integration of sensor fusion techniques and deep learning enabled the system to achieve 99.81% accuracy for dynamic gesture recognition while overcoming limitations commonly associated with vision-based approaches.

Shrestha et al.[6] proposed a CNN-based recognition model using MediaPipe for identifying 50 Nepali Sign Language letters and symbols. The model was trained on more than 125,000 hand landmark images and achieved 99.84% training accuracy along with 99.80% validation accuracy. Additionally, a web-based application supporting real-time finger spelling and consonant-vowel handling in Nepali was implemented.

Basnet et al.[7] developed a smart glove system for translating Nepali Sign Language gestures into text and speech using flex sensors, a 3-axis accelerometer, and a gyroscope. The system employed a Random Forest classifier trained on a custom dataset containing 15,050 samples and achieved recognition accuracies of 75% for alphabets and 80% for words.

Maharjan et al.[8] proposed a smart glove-based sign language translation system equipped with five flex sensors and an MPU-6050 sensor for real-time gesture detection. The system utilized a Recurrent Neural Network (RNN) with Long Short-Term Memory (LSTM) to recognize American Sign Language alphabets and numerals and integrated a text-to-speech module for audio and textual output generation.

Dalal et al.[9] designed a deep learning-based smart glove system incorporating five flex sensors and an MPU-6050 sensor to capture hand motion and orientation data for real-time sign language translation. A Bi-directional Long Short-Term Memory (Bi-LSTM) network trained on a custom dataset of 34 American Sign Language gestures achieved an overall classification accuracy of 82% for both static and dynamic gestures.

Sapkota et al. [10] proposed a smart glove system for sign language translation using flex sensors, touch sensors, and an accelerometer integrated with an Arduino UNO and Bluetooth module. The system translated both American and Nepali Sign Language gestures into text and speech through an Android application while also supporting voice-to-text and text-to-sign conversion. Experimental evaluation showed an overall gesture recognition accuracy of 87.69%, demonstrating the feasibility of two-way communication between speech-impaired individuals and normal users.

Abougarair and Arebi [11] developed a wearable smart glove for translating American Sign Language (ASL) gestures into speech and text using five flex sensors and an ADXL335 accelerometer. The system employed an Arduino Nano and Bluetooth communication to transmit recognized gestures to a mobile application. Experimental results showed an average translation time of 0.6 seconds for gesture-to-text and speech conversion, demonstrating the effectiveness of the proposed wearable communication system.

CHAPTER 3

METHODOLOGY

3.1. Requirement Analysis

3.1.1. Hardware Requirements

1. ESP32

The ESP32 is a low-cost, low-power system-on-chip (SoC) microcontroller with integrated Wi-Fi and dual-mode Bluetooth. It features 34 programmable GPIOs, a 12-bit ADC, 2 8-bit DACs, 4 SPI interfaces, 2 I2S interfaces, 2 I2C interfaces, and 3 UARTs. The device is equipped with 520 KB of RAM and 448 KB of ROM.

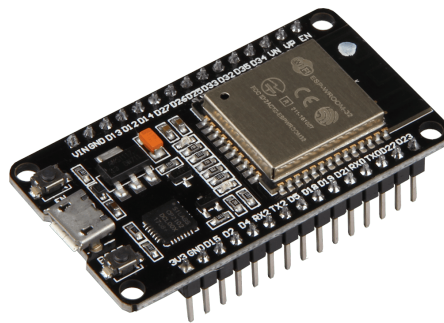


Figure 3.1: ESP32

2. Flex Sensor

A flex sensor, also known as a bend sensor, is a low-cost, easy-to-use sensor that is used to measure the amount of deflection or bending. It is basically a variable resistor, whose resistance varies when bent. The conductive ink on the sensor serves as a resistor. When the sensor is straight, this resistance is around 25K Ω . When the sensor is bent, the conductive layer is stretched, resulting in a reduced cross section (imagine stretching a rubber band) and increased resistance. At a 90° angle, this resistance is approximately 100K Ω . Using this characteristic of the sensor, the amount of bend in the finger is measured.

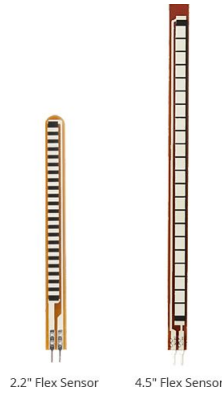


Figure 3.2: Flex Sensor

3. MPU6050

The MPU6050 is a motion-tracking sensor that combines a 3-axis gyroscope and a 3-axis accelerometer, communicating via I2C for easy integration with micro-controllers. It measures both rotational velocity and linear acceleration, enabling precise detection of gestures through six degrees of motion data. Its built-in Digital Motion Processor (DMP) performs onboard calculations, providing fused outputs like quaternions or Euler angles for more accurate motion tracking.

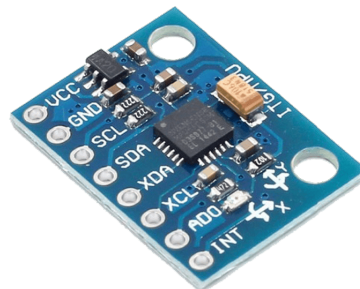


Figure 3.3: MPU6050

3.1.2. Software Requirements

1. Development Environment

- **Visual Studio Code (VS Code):** It is a code editor with features like IntelliSense, syntax highlighting, and a built-in terminal streamlined development. It has been used for end to end system development as a primary code editor for this project.
- **PlatformIO:** An extension within VS Code that simplifies firmware development for ESP32 by managing libraries, dependencies, and board configurations, and offering easy deployment and debugging.

- **Jupyter Notebook:** It is used for interactive Python-based data processing and training machine learning models. It provides data visualization, pre-processing, model training, and performance evaluation, supporting rapid experimentation.
- **KiCad:** KiCad is an open-source software suite for Electronic Design Automation (EDA). It provides tools for schematic capture and printed circuit board (PCB) layout, supporting industry-standard outputs like Gerber and IPC-2581 files. KiCad is compatible with Windows, Linux, and macOS, and is distributed under the GNU General Public License version 3 (GPL v3).

2. Programming Languages, Libraries and Packages

- **C++ (Arduino Framework):** It is used to develop firmware for the ESP32, allowing control over hardware components like flex sensors and MPU6050 via serial communication and digital I/O.
- **Python:** Python is an open-source language used for data handling, machine learning, deep learning and visualization with rich libraries support like scikit-learn, TensorFlow, Pytorch, Matplotlib, and Seaborn that can be utilized for building and evaluating gesture recognition models.
- **Numpy and Pandas:** These are used for data collection, labeling and pre-processing. Data is collected in csv format. NumPy handles numerical operations and filtering, while Pandas structures the sensor data into labeled DataFrames for easier manipulation and analysis.
- **Scikit-learn:** Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data pre-processing, model selection, model evaluation, and many other utilities.[12]
- **TensorFlow and Keras:** TensorFlow is an open-source deep learning framework developed by Google, helps to build and train neural networks efficiently across CPU and GPU environments. Keras serves as its high-level API, providing a simplified and readable interface for designing neural network architectures with minimal code.
- **gTTS:** gTTS is a lightweight Python library that interfaces with Google's Text-to-Speech API to convert written text into spoken audio output.

3.2. Block Diagram

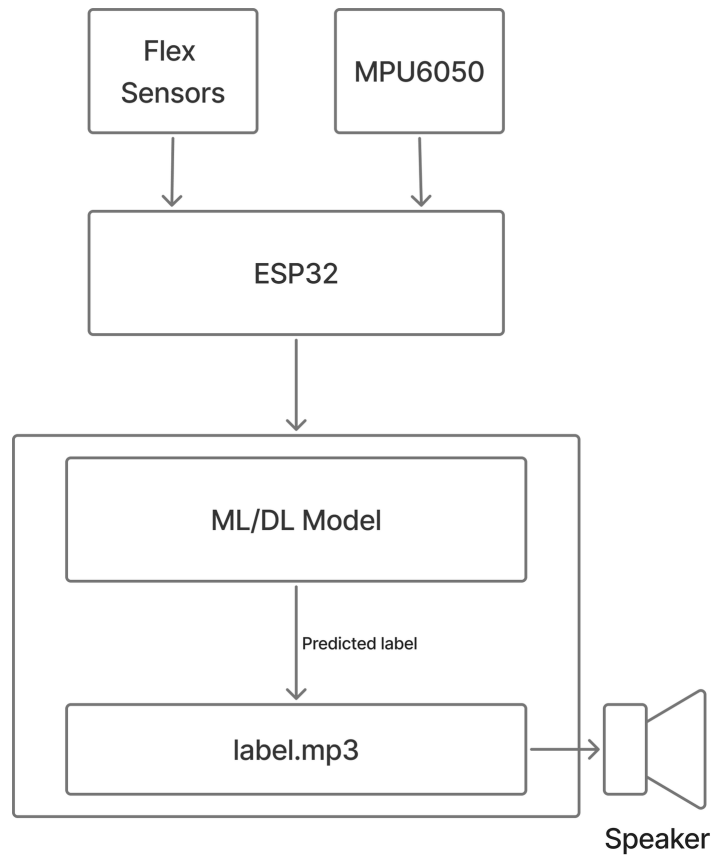


Figure 3.4: Block Diagram

The figure shows high level block diagram of the system, the flex sensors and MPU6050 sensor are used to capture finger bending and hand movement data, respectively from the glove. These sensor readings are sent to the ESP32, which acts as the main controller of the system. The ESP32 forwards the collected data to the ML/DL model for gesture recognition. After processing the sensor inputs, the model predicts the corresponding Nepali sign label. Based on the predicted label, the related pre-generated audio file (label.mp3) is played through the speaker, allowing the recognized Nepali sign gesture to be converted into real-time speech output.

3.3. Hardware Interfacing

3.3.1. Interfacing of Flex Sensors

The glove consists of a total of 8 flex sensors, one sensor each for thumb and little finger, two sensors each for other fingers placed to read the data from the upper and lower joints. Here, each sensor is connected to an ESP32 ADC pin to provide input data.

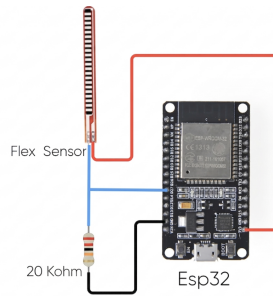


Figure 3.5: Flex Sensor Connection

Sensor Placement: To accurately capture finger bends and movements, flex sensors are placed on specific finger joints to measure how much each joint bends. In this work, the proximal interphalangeal (PIP) joints are labeled as “Up,” and the metacarpophalangeal (MCP) joints are labeled as “Low.” This naming system helps keep the dataset organized and makes feature labels easier to understand. For the index, middle, and ring fingers, two sensors are used per finger one placed across the PIP joint (idxUp, midUp, ringUp) and another placed across the MCP joint (idxLow, midLow, ringLow) where as one sensor each for thumb and little finger. Each sensor is aligned along the finger so that it bends naturally with the joint.

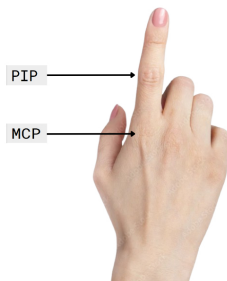


Figure 3.6: Joint

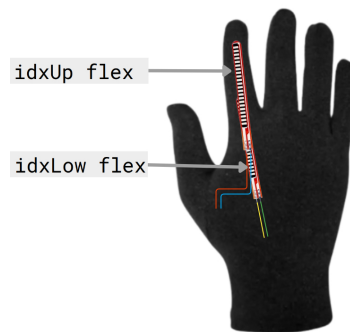


Figure 3.7: Sensor Placement

Selection of Resistor: The flex sensors used in the project have an approximate resistance of $20\text{ k}\Omega$ in the unflexed state and $24\text{ k}\Omega$ when flexed. To convert this resistance variation into a measurable voltage, each sensor was connected in a voltage divider configuration with a fixed series resistor (R_2). The resistor value was selected analytically to maximize the voltage difference between the two sensor states, improving ADC sensitivity and measurement reliability. The voltage response of the divider circuit was evaluated for resistor values ranging from $1\text{ k}\Omega$ to $200\text{ k}\Omega$. Lower resistor values produced higher output voltages but smaller voltage variation between flexed and unflexed conditions. Increasing the resistor value reduced the overall output voltage while maintaining distinguishable separation between the two states. The analysis showed that resistor values between approximately $20\text{ k}\Omega$ and $100\text{ k}\Omega$ provided suitable voltage spacing for accurate analog-to-digital conversion.

Considering both sensitivity and practical component availability, a $20\text{ k}\Omega$ resistor is used for the final design. This value offered stable sensor readings with sufficient voltage variation between flexed and unflexed states, making it suitable for reliable gesture data acquisition.

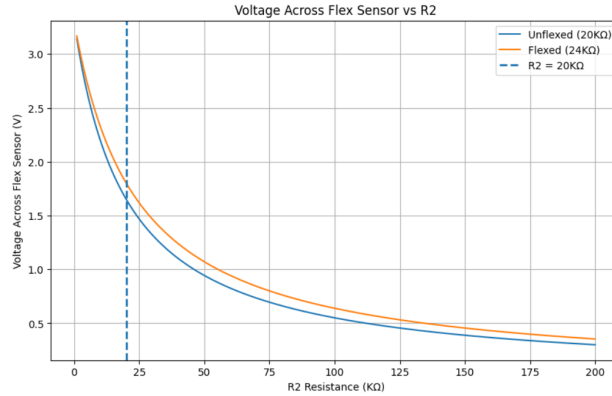


Figure 3.8: Voltage across flex sensor vs Series Resistor

3.3.2. Interfacing of IMU

The MPU6050 IMU is interfaced with the ESP32 using the I2C communication protocol. The VCC and GND pins of the MPU6050 are connected to the 3.3V supply and ground of the ESP32, while the SDA and SCL pins are connected to the ESP32 I2C pins, GPIO21 and GPIO22 respectively.

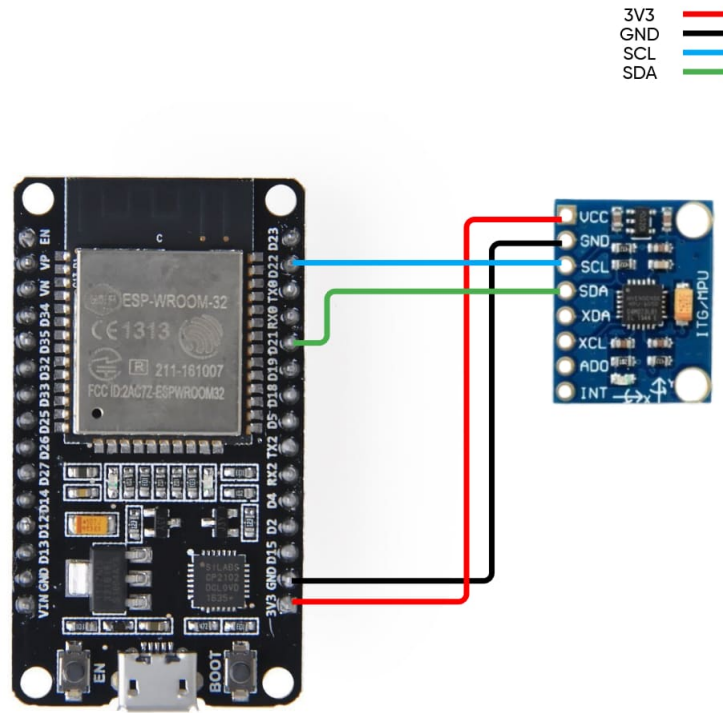


Figure 3.9: Interfacing of MPU6050

3.4. Sensor Calibration

The flex sensors used in this project presented a notable hardware limitation related to their physical construction. When the glove is left unused for a period of time, the ink inside the flex sensors tends to settle and harden. To restore normal sensor function, the user should wear the glove and perform 15–20 repetitions of opening and closing the fist. This action redistributes the ink particles and brings the sensors back to their optimal working condition.

At startup, the system performs an automatic baseline calibration where all sensors record 100 readings while the hand remains in a neutral position. The average of these readings is stored as each sensor's baseline and is later used to normalize bending measurements.

3.5. Data Collection and Dataset Preparation

3.5.1. Data Collection From Hardware

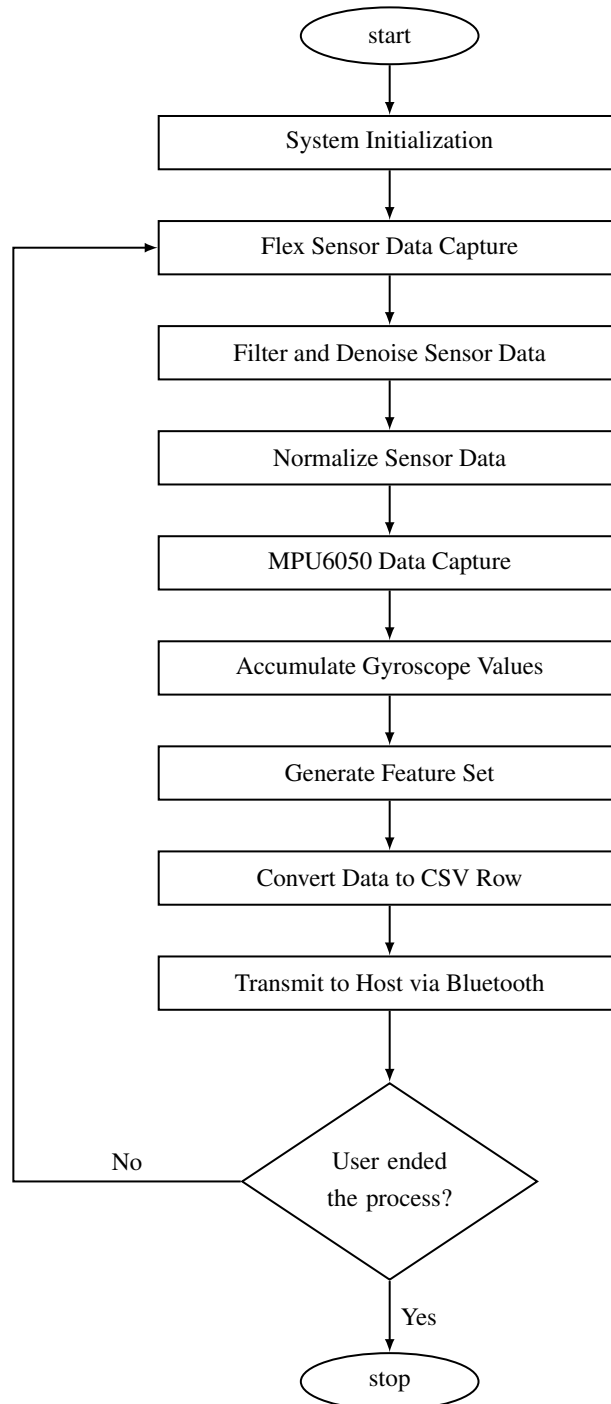


Figure 3.10: Flowchart for data acquisition process.

Upon powering on, the ESP32 initializes all connected flex sensors and the MPU6050 IMU sensor. Once initialized, the system continuously captures real-time hand gesture data comprising both finger bending and hand motion measurements.

The flex sensors measures finger joint bending angles, and the raw readings were filtered to remove electrical noise and jitter. The filtered values were then normalized and mapped to a fixed range of 0–1000 to ensure consistency across varying hardware and environmental conditions. Concurrently, the MPU6050 communicates with the ESP32 via the I2C interface, providing accelerometer and gyroscope measurements along the x , y , and z axes. Cumulative gyroscope values (acc_gx , acc_gy , acc_gz) were computed by accumulating angular motion over time to capture temporal patterns necessary for dynamic gesture recognition. The sensor readings are sampled every 60 ms. All sensor readings were then combined into a single feature vector per sample, organized in the following format:

$$(idxUp, idxLow, midUp, midLow, ringUp, ringLow, thumb, pinky, ax, ay, az, gx, gy, gz, acc_gx, acc_gy, acc_gz)$$

This feature vector was subsequently serialized into a CSV row and transmitted to the host system via serial communication for dataset creation and further processing.

Table 3.1: Sample of Raw CSV Data Transmitted from the ESP32 Hardware During the Data Collection Process

idxUp	idxLow	midUp	midLow	ringUp	ringLow	thumb	pinky	ax	ay	az	gx	gy	gz	acc_gx	acc_gy	acc_gz
156	46	131	64	240	34	98	160	-0.17	0.72	0.67	0	2	-1	6.85	5.85	-6.69
154	48	130	64	242	34	98	158	-0.17	0.72	0.67	0	2	-1	6.74	6.14	-6.81
154	48	130	64	242	33	100	158	-0.17	0.72	0.67	0	2	-1	6.74	6.22	-6.81
156	48	129	63	240	34	100	155	-0.17	0.72	0.67	-1	2	-1	6.65	6.33	-6.87
154	48	128	63	240	33	100	155	-0.17	0.72	0.67	-1	2	-1	6.59	6.39	-6.90
154	48	129	63	240	33	100	153	-0.17	0.72	0.66	-1	2	-1	6.54	6.46	-6.93
154	48	128	63	237	33	100	153	-0.17	0.72	0.66	-1	2	-1	6.47	6.64	-7.00
154	48	128	63	237	33	103	153	-0.17	0.72	0.66	-1	2	-1	6.47	6.77	-7.00
151	48	126	63	234	33	100	153	-0.17	0.72	0.67	-1	3	0	6.43	6.96	-7.06
151	48	126	63	234	33	103	153	-0.17	0.72	0.67	0	2	-1	6.33	6.96	-7.15
151	48	126	63	234	33	103	153	-0.17	0.72	0.67	-1	2	-1	6.29	6.96	-7.23

3.5.2. Dataset Creation for Static Gestures

During the static gesture data collection phase, a Python program was developed to receive sensor data transmitted from the ESP32 via Bluetooth. The incoming data, organized in CSV format, contained flex sensor readings, accelerometer values, gyroscope values, and accumulated gyroscope parameters (gx , gy , and gz , acc_gx , acc_gy , and acc_gz). Since the gyroscope values and accumulated gyroscope values are mainly required for dynamic gesture recognition, they were excluded during static dataset creation. Only the flex sensor readings, accelerometer values, and instantaneous gyroscope readings were retained for static gesture classification.

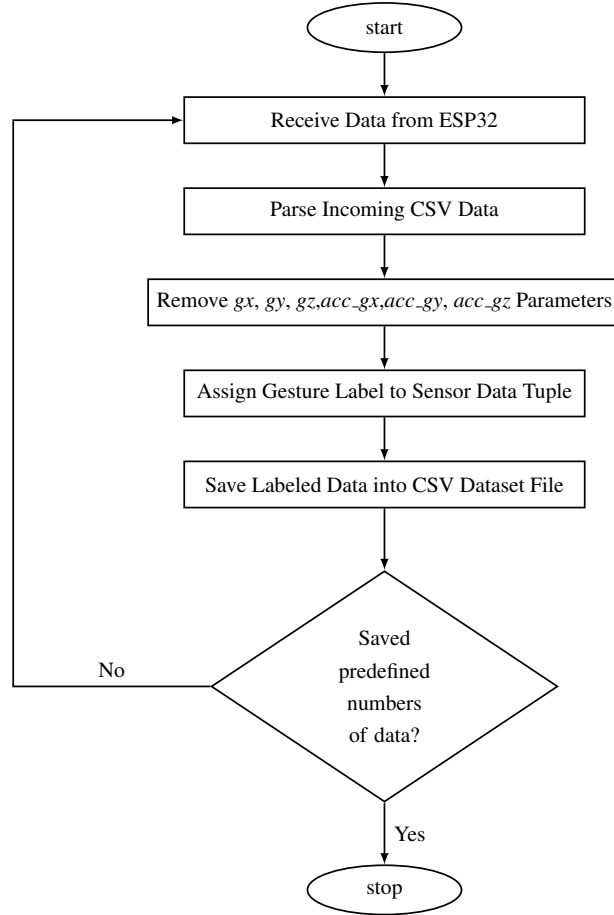


Figure 3.11: Flowchart of the static gesture dataset creation process.

The static dataset was created manually by repeatedly performing predefined Nepali Sign Language gestures while recording the corresponding sensor values. The selected gestures included four Nepali alphabet gestures: *ka*, *kha*, *ga*, *gha*, and *nga* along with four numerical gestures: *ek*, *dui*, *tin*, and *char*. 1000 samples were collected for each gesture making dataset of 9000 samples. After data acquisition, each sensor data tuple was assigned its corresponding gesture label and appended to a CSV file.

Table 3.2: Sample Records from the Static Gesture Dataset

idxUp	idxLow	midUp	midLow	ringUp	ringLow	thumb	pinky	ax	ay	az	label
88	64	169	145	388	144	79	319	-0.31	0.88	0.31	ka
0	0	169	123	345	126	0	309	-0.15	0.77	0.59	kha
84	0	168	97	377	147	110	324	-0.39	0.76	0.49	ga
0	0	0	0	77	51	115	70	-0.95	0.20	0.07	gha
169	42	174	115	357	113	0	37	-0.95	0.22	-0.01	nga
0	0	157	84	320	94	74	299	-0.61	0.75	0.18	ek
0	0	5	7	311	77	28	274	-0.62	0.74	0.19	dui
0	0	5	0	145	50	38	251	-0.45	0.83	0.21	tin
0	0	10	0	122	54	115	128	-0.48	0.83	0.16	char

3.5.3. Dataset Creation for Dynamic Gestures

Similarly, for dynamic gesture data collection, another Python program was developed to continuously receive sensor data transmitted from the ESP32. To identify whether the hand movement was static or dynamic, the program maintained a rolling buffer containing the latest 20 sensor samples. The difference between the first and most recent accumulated gyroscope values was calculated along the X, Y, and Z axes. If the change in any axis exceeded a threshold value of 20, the motion was considered dynamic; otherwise, it was treated as static.

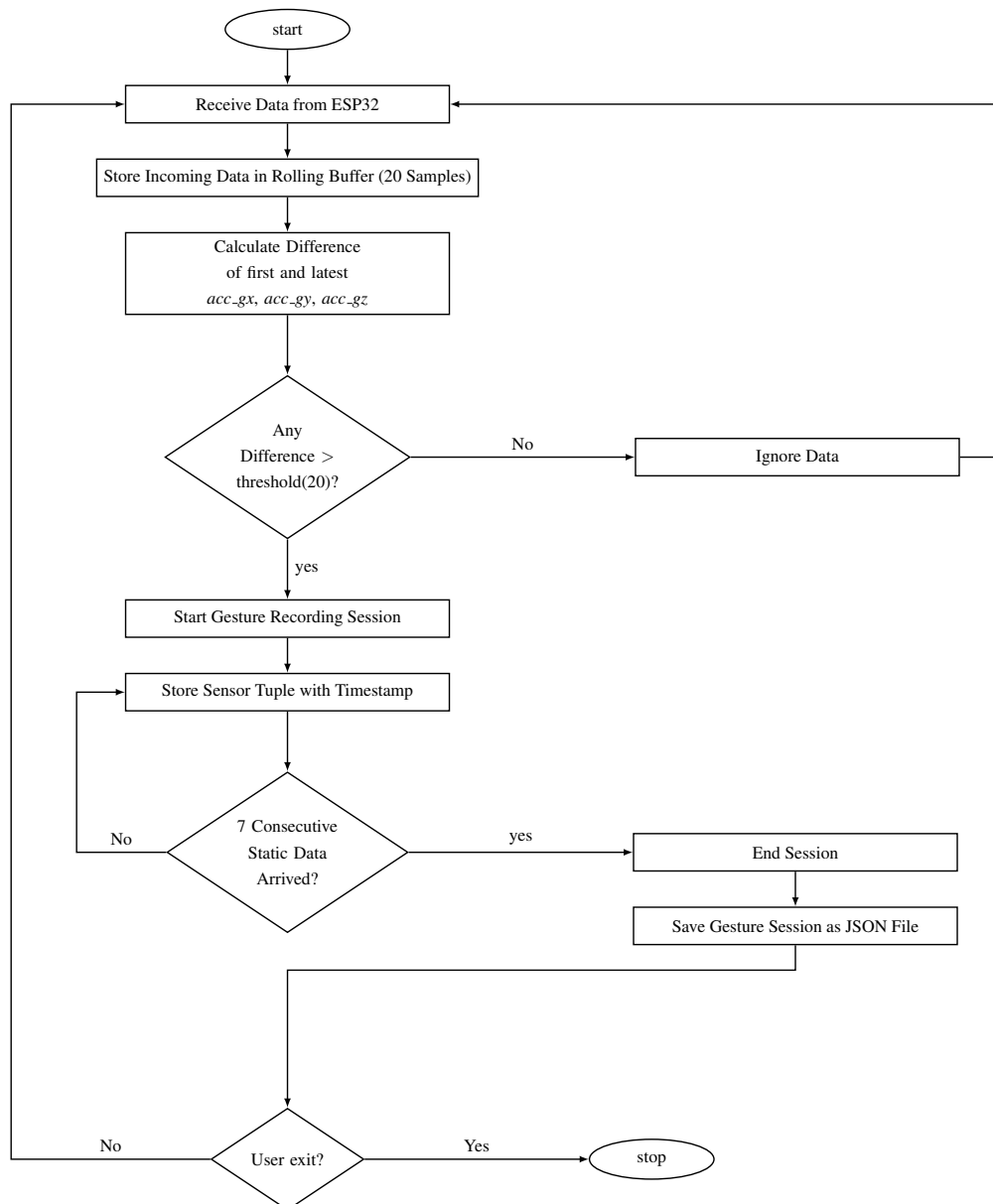


Figure 3.12: Flowchart of the dynamic gesture dataset creation process.

During the idle state, static samples were ignored to avoid storing unnecessary data. Once a dynamic motion was detected, the system started a recording session and con-

tinuously stored incoming sensor readings together with their timestamps. The recording continued until seven consecutive static samples were observed, which indicated the end of the gesture. The captured gesture sequence was then saved in JSON format along with its gesture label and timestamped sensor values. After storing the data, the program returned to the monitoring state and waited for the next gesture until the session was stopped manually.

The dynamic gesture dataset was also collected manually by repeatedly performing selected Nepali Sign Language word level gestures total of 7 “namaste”, ”malai”, ”chinnu”, ”mero”, ”naam”, ”sanjog”, ”ho”, so that a basic introduction can be demonstrated. Each gesture was performed and recorded 200 times manually. The data augmentation technique was used to increase the dataset size by adding some white noise in the sensor values. 200 recordings for each gesture were converted to 1500 for each to create a sufficiently diverse dataset for training and evaluation.

3.6. Model Training and Development

3.6.1. Random Forest for Static Gesture

Random Forest is a supervised machine learning algorithm that constructs an ensemble of decision trees to improve prediction performance. Each tree is trained on a randomly sampled subset of the data, and the final prediction is obtained through majority voting in classification tasks. The behaviour of a Random Forest model can be controlled by tuning several hyperparameters, such as `n_estimators`, which determines the number of trees in the forest, and `max_depth`, which specifies the maximum depth allowed for each tree.

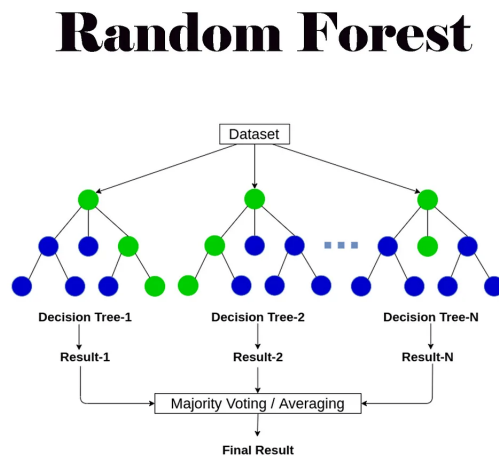


Figure 3.13: Random Forest

Since the features span very different scales (finger position values reach 369 while IMU readings like a_x/a_y sit between -1 and 1), `StandardScaler` was applied to normalize everything to zero mean and unit variance. The label column was encoded using `LabelEncoder`. `GridSearchCV` technique was used to find the best hyperparameters to train the model. The dataset was split into 70% for training and 30% for testing. The final trained model with hyperparameters `n_estimator=30`, `max_sample=0.70` was selected for the deployment.

3.6.2. LSTM for Dynamic Gesture

Unlike static gestures, dynamic gestures involve continuous hand movement, orientation changes, and temporal motion patterns. In such cases, individual sensor readings alone are insufficient because the meaning of the gesture depends on the sequence of movements over time. Traditional machine learning models cannot effectively capture temporal dependencies between consecutive sensor readings. Therefore, LSTM, a recurrent deep learning architecture designed for sequential data processing, was selected for dynamic gesture recognition.

LSTM is a specialized form of Recurrent Neural Network (RNN) capable of learning long-term temporal dependencies from sequential data. Unlike standard RNNs, LSTM networks use memory cells and gating mechanisms to control information flow through the network. These include the input gate, forget gate, and output gate, which selectively retain or discard information over time. This architecture enables LSTM models to effectively learn motion patterns, transitions, and sequential dependencies present in dynamic gestures.

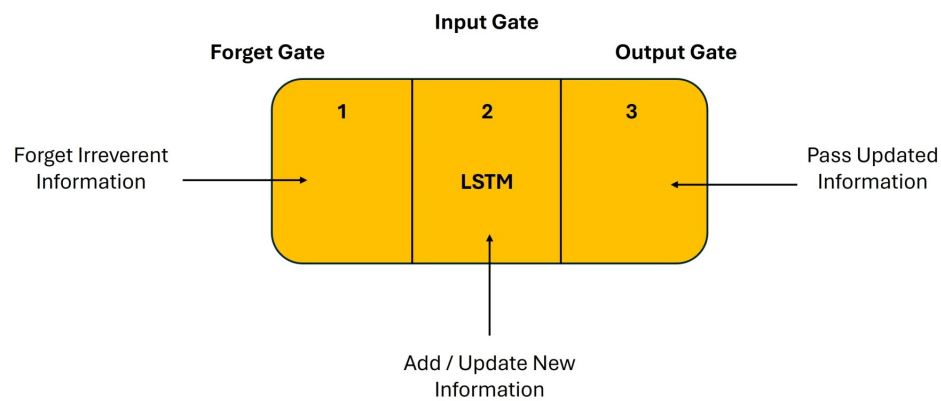


Figure 3.14: LSTM

Data Preprocessing

Before training the model, several preprocessing steps were performed. First, all gesture recordings stored in JSON format were loaded and validated. Very short record-

ings containing insufficient frames were discarded to avoid unreliable learning patterns. From each frame, fifteen features were extracted, including eight flex sensor readings, three accelerometer values, three gyroscope values, and one timestamp feature. The timestamp was normalized between 0 and 1 so that the model could learn the relative timing pattern of the gesture instead of depending on the actual performing speed.

Since different users performed gestures at different speeds, the recorded sequences contained varying numbers of frames. To solve this issue, all gesture sequences were resampled to a fixed length of 70 frames using linear interpolation. This ensured that every sample had the same input dimensions before being passed to the LSTM network. After resampling, Min-Max normalization was applied to all features to scale the values within a common range. Finally, gesture labels were converted into numerical encoded values and transformed into one-hot vectors for multi-class classification.

Model Training

The dataset was split into 80% for training and 20% for testing. A 2 layer LSTM architecture was trained by experimentally adjusting different hyperparameters such as hidden unit sizes, dropout rates, batch sizes to achieve stable training performance. The final trained model with 16 units in each layer, 0.5 dropout rate and batch size of 32 was selected for the deployment. The maximum epoch count was set to 80. However, Early Stopping was implemented to automatically terminate training when validation loss stopped improving for ten consecutive epochs. This was used to prevent unnecessary training and possibility of overfitting.

```

263/263 ————— 3s 13ms/step - accuracy: 0.9412 - loss: 0.1249 - val_accuracy: 0.9410 - val_loss: 0.1211
Epoch 35/80
263/263 ————— 3s 13ms/step - accuracy: 0.9404 - loss: 0.1294 - val_accuracy: 0.9424 - val_loss: 0.1151
Epoch 36/80
263/263 ————— 6s 17ms/step - accuracy: 0.9436 - loss: 0.1172 - val_accuracy: 0.9381 - val_loss: 0.1169
Epoch 37/80
263/263 ————— 3s 13ms/step - accuracy: 0.9412 - loss: 0.1260 - val_accuracy: 0.9381 - val_loss: 0.1125
Epoch 38/80
263/263 ————— 3s 13ms/step - accuracy: 0.9420 - loss: 0.1209 - val_accuracy: 0.9367 - val_loss: 0.1130
Epoch 39/80
263/263 ————— 4s 14ms/step - accuracy: 0.9431 - loss: 0.1212 - val_accuracy: 0.9357 - val_loss: 0.1268
Epoch 40/80
263/263 ————— 4s 15ms/step - accuracy: 0.9398 - loss: 0.1218 - val_accuracy: 0.9367 - val_loss: 0.1157
Epoch 41/80
263/263 ————— 3s 13ms/step - accuracy: 0.9356 - loss: 0.1433 - val_accuracy: 0.9371 - val_loss: 0.1159
Epoch 42/80
263/263 ————— 3s 13ms/step - accuracy: 0.9402 - loss: 0.1287 - val_accuracy: 0.9410 - val_loss: 0.1151
Epoch 43/80
263/263 ————— 4s 17ms/step - accuracy: 0.9398 - loss: 0.1239 - val_accuracy: 0.9405 - val_loss: 0.1134
Epoch 44/80
263/263 ————— 3s 13ms/step - accuracy: 0.9405 - loss: 0.1171 - val_accuracy: 0.9381 - val_loss: 0.1595
Epoch 45/80
263/263 ————— 3s 13ms/step - accuracy: 0.9436 - loss: 0.1185 - val_accuracy: 0.9362 - val_loss: 0.1160
Epoch 46/80
263/263 ————— 4s 14ms/step - accuracy: 0.9419 - loss: 0.1187 - val_accuracy: 0.9367 - val_loss: 0.1184
Epoch 47/80
263/263 ————— 4s 15ms/step - accuracy: 0.9414 - loss: 0.1183 - val_accuracy: 0.9395 - val_loss: 0.1139
Test Accuracy: 93.81%
66/66 ————— 1s 6ms/step

```

Figure 3.15: LSTM Training

3.7. Real-Time Gesture Prediction Pipeline

These sensor values are continuously transmitted from the ESP32 to the host computer through via Bluetooth. After receiving the data, a Python inference script performs preprocessing, which includes formatting the sensor readings, filtering noise, and creating the required feature vector. The system then performs motion detection using gyroscope data. A rolling buffer of recent gyroscope readings is maintained, and the difference between the oldest and latest readings is calculated. If the gyroscope variation exceeds a predefined threshold, the gesture is considered dynamic; otherwise, it is treated as static.

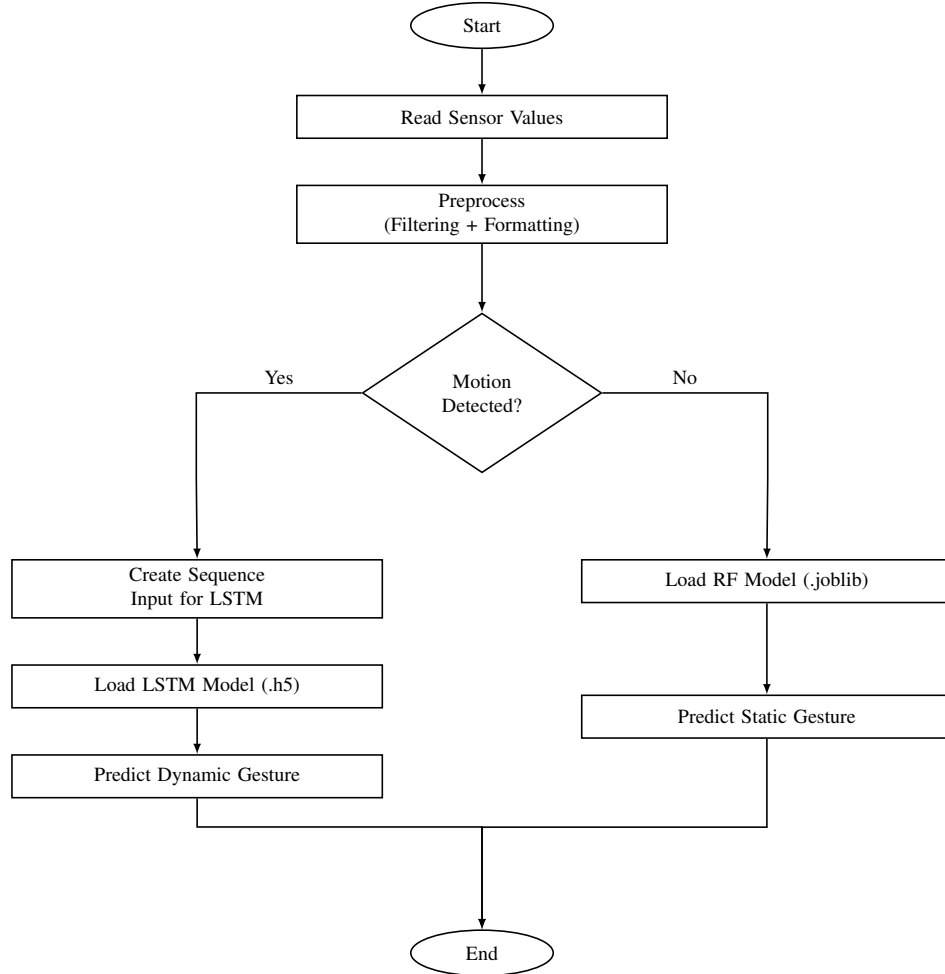


Figure 3.16: Flowchart of the real-time gesture prediction pipeline.

For static gestures, the current sensor frame is directly passed to the trained Random Forest model stored in `.joblib` format. The model predicts the corresponding static gesture label such as *ka*, *kha*, or numbers. For dynamic gestures, multiple sequential sensor frames are accumulated to form a time-series sequence. This sequence is then preprocessed through resampling and normalization before being fed into the trained LSTM model stored in `.h5` format and, it predicts the dynamic gesture.

3.8. Text-to-Speech Conversion

Each gesture label, both static and dynamic, was pre-converted into a Nepali audio file using the gTTS library and stored locally. Once a gesture label is predicted, the corresponding audio is retrieved and played through the laptop speaker.

CHAPTER 4

RESULT AND ANALYSIS

4.1. Real-Time Prediction Results

The overall system was evaluated on a small sample of real-time predictions across all trained 16 gestures both static and dynamic. Each gesture was performed 20 times, and the number of correct and incorrect predictions were noted. The system achieved an overall accuracy of approximately **93.1%** on this testing sample. Also the average translation time after completion of static gesture was found to be 0.5 sec, and for dynamic it was about 2 sec .

Table 4.1: Real-Time Prediction Results for Each Gesture (20 Trials Each)

Gesture	Correctly Predicted	Incorrectly Predicted
Namaste	15	5
Mero	16	4
Naam	20	0
Sanjog	18	2
Ho	20	0
Malai	18	2
Chinnu	19	1
ek	19	1
dui	20	0
tin	20	0
char	17	3
ka	20	0
kha	20	0
ga	18	2
gha	18	2
nga	20	0
Total	298	22

4.2. Analysis of Random Forest Model

The Random Forest model achieved a training accuracy of 99.94%, test accuracy of 99.56%, and an OOB score of 99.62%. These results indicate that the model was able to classify the static gestures with very high accuracy. The small difference between training and test accuracy shows that the model generalized well on unseen data and did not suffer from significant overfitting. Similarly, the high OOB score further confirms the reliability and stability of the model during prediction.

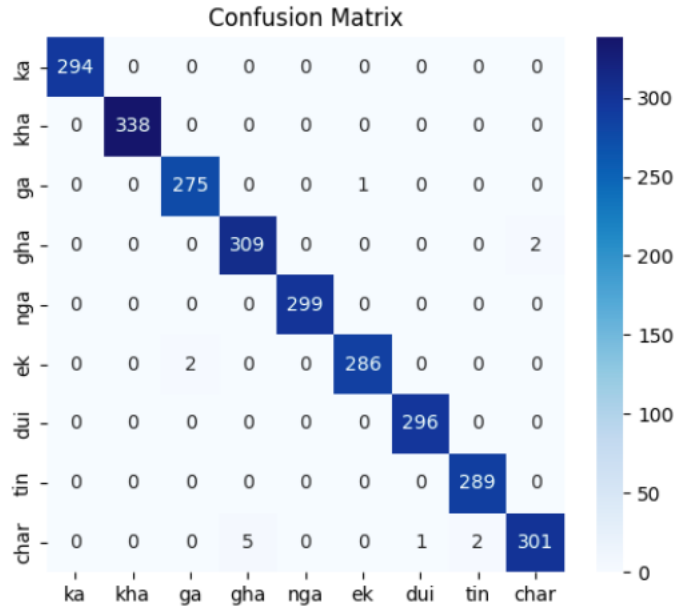


Figure 4.1: Confusion Matrix of Random Forest Model

4.3. Analysis of LSTM model

The confusion matrix shows that the LSTM model was able to correctly classify most of the dynamic gesture samples with high accuracy. Gestures such as *ho*, *malai*, and *naam* were predicted almost perfectly, while a small number of misclassifications occurred between similar gestures such as *mero* and *namaste*, and between *chinnu* and *sanjog*.

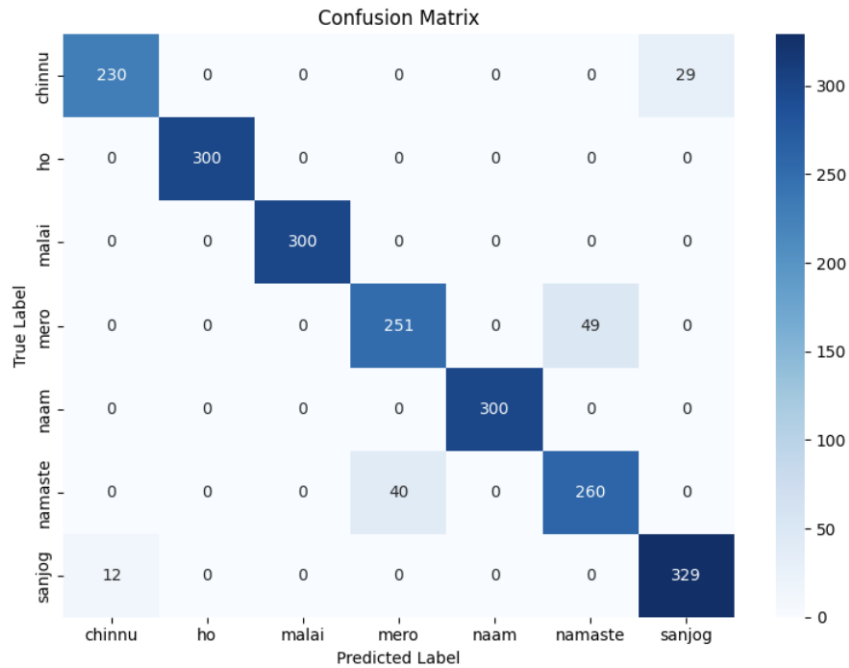


Figure 4.2: Confusion Matrix of LSTM Model

Similarly, the learning curve shows that both training and validation accuracy increased rapidly during the initial epochs and gradually stabilized around 94%. The close alignment between the training and validation curves indicates that the model generalized well on unseen data and did not suffer from significant overfitting.

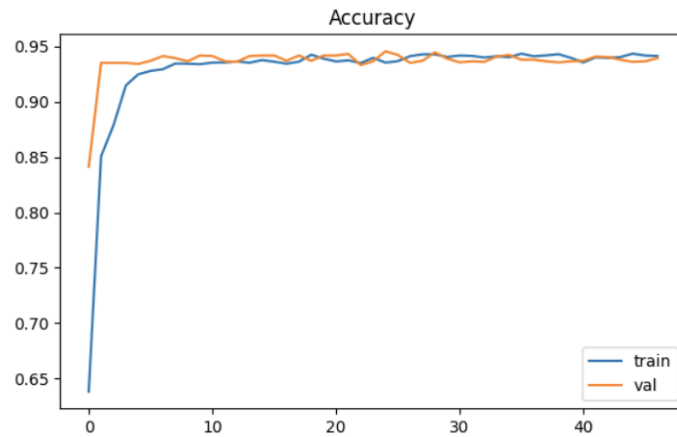


Figure 4.3: Learning Curve

An analysis of the model shows that it largely overlooks the first 46 steps of a sequence, focusing instead on the final moments, where importance scores peak sharply between steps 66 and 69. By prioritizing these "ending signatures" to make its predictions, the LSTM network becomes highly sensitive to how a sequence finishes. While this temporal bias generally yields high accuracy, it also explains why the model confuses "mero" and "namaste" at the final stage of the sequence, their patterns look remarkably similar to the network. While the current implementation achieves satisfactory accuracy, it requires corrective measures. Several approaches have been identified that could directly address this issue. An attention mechanism into the LSTM architecture or Bidirectional LSTM can address this problem.

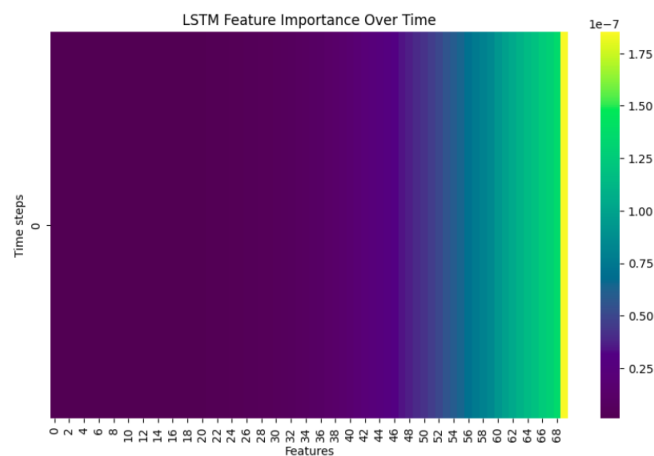


Figure 4.4: LSTM Feature Importance Over Time

CHAPTER 5

EPILOGUE

5.1. Challenges and Limitations

During the development of this project we faced several challenges and identified some limitations. One of the major challenges was proper sensor placement and calibration. Since each finger bends differently, positioning the flex sensors accurately on the glove was difficult. Even small changes in sensor placement affected the readings. In addition, the flex sensors required careful calibration to obtain stable and consistent values as the ink used in the sensor hardens if not used for a period of time, 15–20 repetitions of opening and closing the fist should be performed to redistribute the ink particles and brings the sensors back to their optimal working condition.

Another challenge was managing separate datasets for static and dynamic gestures. Since the project uses two different models, the collected data had to be stored and organized separately according to the type of gesture. Maintaining proper labeling, sequence formatting, and storage structure for both datasets required additional effort during dataset creation and preprocessing.

During model training, the small dataset size resulted in very high training and testing accuracy, which made it difficult to verify whether the models were truly learning the gesture patterns or simply overfitting the limited data. Multiple evaluations and testing approaches were required to ensure that the trained models could generalize reasonably well during real-time prediction.

Developing the real-time prediction pipeline was also challenging. A separate logic had to be designed to continuously analyze incoming sensor values and determine whether the gesture was static or dynamic. Based on this decision, the correct model had to be loaded and executed in real time. Ensuring smooth switching between models while maintaining correct gesture prediction and avoiding false outputs required careful program design and testing.

The current prototype also has several limitations. The project currently supports only a limited number of gestures and the dataset was collected from a small number of users. Due to this, the trained models may not generalize well across different users, hand sizes, or signing styles. In addition, the glove contains multiple sensors and connecting wires, making the hardware slightly bulky and less comfortable for prolonged use. Speech translation time for dynamic averages around 2 seconds, which may feel sluggish for real-time conversation use cases

5.2. Future Enhancements

Several improvements can be made in the future to enhance the performance and usability of the system. One important direction is expanding the dataset by collecting data from actual deaf community under different conditions. Current implementation show the LSTM model make the predictions by prioritizing the ending pattern of the gesture. Advanced models such as an attention mechanism into the LSTM architecture or Bidirectional LSTM can be used to address this problem.

Future work can also focus on supporting a larger vocabulary of Nepali Sign Language gestures, including complex dynamic gestures, sentence-level recognition, and two-handed signs. Incorporating additional IMUs or advanced motion-tracking sensors may further improve motion capture accuracy and spatial understanding of gestures.

Another possible enhancement is optimizing deep learning models for embedded deployment so that real-time inference can run directly on the ESP32 or similar edge devices. A mobile application can be developed to enhance portability and user experience by enabling real-time speech output directly from a smartphone. A major future work can be minimizing the the latency of speech output.

5.3. Conclusion

The prototype demonstrates the feasibility of real-time Nepali Sign Language recognition using sensors and machine learning techniques. The system successfully captures finger bending and hand motion data using flex sensors and an MPU6050 IMU for recognizing both static and dynamic gestures. The project also integrates pre-generated speech output using a text-to-speech system, used for real-time gesture to speech conversion. Experimental results showed that the proposed approach can effectively classify gestures within the limited vocabulary used in the current implementation.

Although the system still has limitations related to dataset size, gesture coverage, and hardware constraints, the work highlights the potential of combining wearable sensing, machine learning, and deep learning for practical sign language translation systems. With further improvements in data collection, model optimization, the system can be extended into a more robust and scalable assistive communication solution in the future.

APPENDIX A

A.1. Prototype Glove



Figure A.1: Prototype Glove

A.2. PCB Schematic

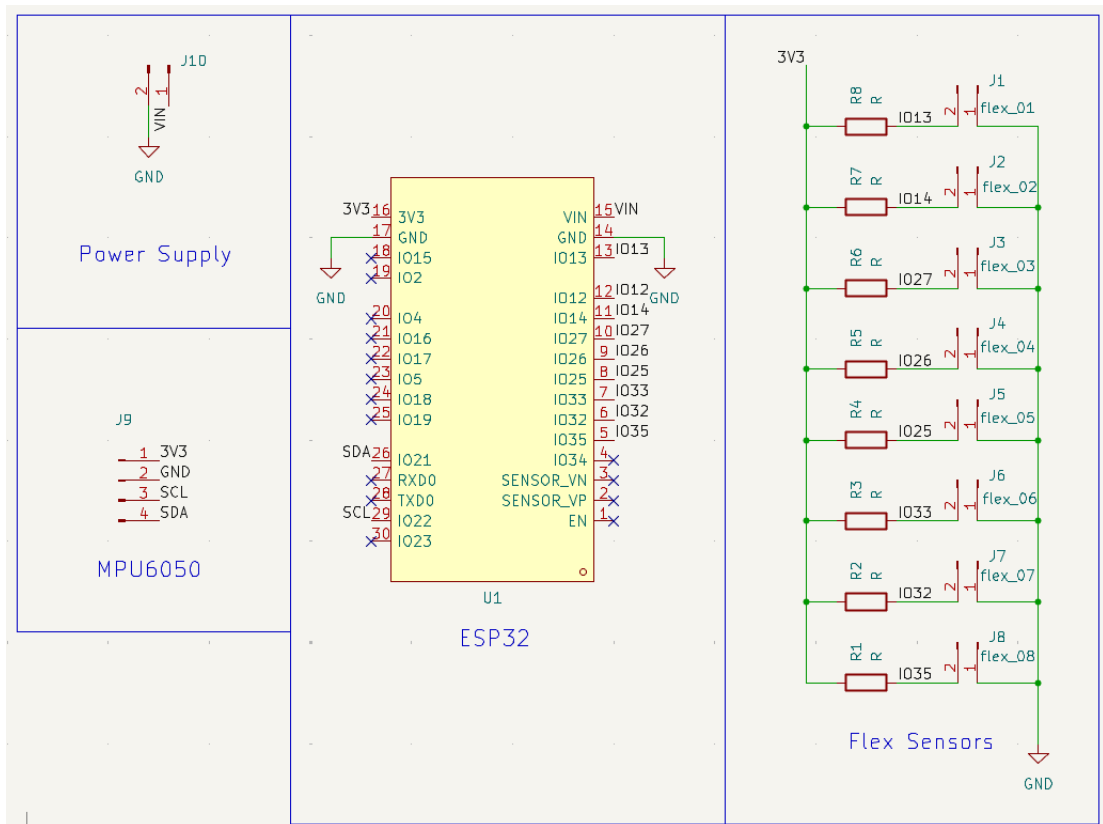


Figure A.2: PCB Schematic

A.3. Example Gestures



Figure A.3: Example Gestures

REFERENCES

- [1] D. Bhusal and D. S. Baral, "Sentence-level nepali sign language recognition," in *Conference Paper*, 2024.
- [2] A. Aryal, B. Gyawali, D. Aryal, R. Shrestha, and H. K.C., "Design and implementation of smart glove for sign language vocalization," *International Research Journal of Innovations in Engineering and Technology (IRJIET)*, vol. 6, no. 6, pp. 94–98, Jun. 2022. [Online]. Available: <https://doi.org/10.47001/IRJIET/2022.606012>
- [3] K. Xia, J. Huang, and H. Wang, "Lstm-cnn architecture for human activity recognition," *IEEE Access*, vol. 8, pp. 56 855–56 866, 2020.
- [4] J. Wu, P. Ren, B. Song, R. Zhang, C. Zhao, and X. Zhang, "Data glove-based gesture recognition using cnn-bilstm model with attention mechanism," *PLOS ONE*, vol. 18, no. 11, p. e0294174, Nov. 2023. [Online]. Available: <https://doi.org/10.1371/journal.pone.0294174>
- [5] B. G. Lee, T.-W. Chong, and W.-Y. Chung, "Sensor fusion of motion-based sign language interpretation with deep learning," *Sensors*, vol. 20, no. 21, p. 6256, 2020.
- [6] G. Shrestha, N. Thing, P. Dhakal, P. Dahal, P. Karki, and M. K. Guragai, "Nepali sign language letter detection and finger spelling using mediapipe and cnn," in *Proceedings of 15th IOE Graduate Conference*, vol. 15. Institute of Engineering, Tribhuvan University, May 2024, pp. 166–170.
- [7] A. Ghimire, B. Kadayat, A. Basnet, and A. Shrestha, "Smart glove," in *Conference Paper*. Kathmandu Engineering College, 2019.
- [8] S. Maharjan, S. Shrestha, and S. Fernando, "Sign language detection and translation using smart glove," *Computer Networking, Security & Data Analytics*, 2024, sS_23.
- [9] U. Dalal, A. Thadhani, M. Upadhye, S. Shah, M. Narvekar, and N. Patil, "Deep learning-enabled smart glove for real-time sign language translation," *Journal of Electrical Systems*, vol. 20, no. 10s, pp. 4874–4882, 2024.
- [10] B. Sapkota, M. K. Gurung, P. Mali, and R. Gupta, "Smart glove for sign language translation," in *Proceedings of IEEE Conference*, Lalitpur, Nepal, 2019.

- [11] A. J. Abougarair and W. A. Arebi, “Smart glove for sign language translation,” *International Robotics & Automation Journal*, vol. 8, no. 3, pp. 109–117, 2022.
- [12] Scikit-learn Developers, “Getting started — scikit-learn 1.8.0 documentation,” https://scikit-learn.org/stable/getting_started.html, 2026, accessed: 2026-05-14.